BLACKDUCK®

# IAST Demystified:
## Under the Hood of Interactive Application Security Testing.

Ira Cherkes-Levinshteyn, Staff Engineer, Black Duck.
Katy Sidorova, Principal Software Engineer, Dynatrace.

10.0.2025

# Hello!

**I am Ira Cherkes Levinshteyn**

I am a staff engineer at Black Duck, working on Seeker, an Interactive Application Security Testing (IAST) solution.

I use instrumentation in Java to help developers secure their applications.

I'm also a coach for women in tech, and a proud mother of three girls.

My hobby is hiking in the mountains and desert.

You can find me at in / f ira-cherkes-levinshteyn

# Hello!

## I am Katy Sidorova

I am a principal engineer at Dynatrace, working on a product that helps software developers make more efficient and secure software.

I use instrumentation in Java, Python and other languages to develop our product.

I spend most of my free time travelling, either in Israel or abroad. My last big trip was to China and I arranged everything myself. My other hobby is mixology - making elaborate cocktails.

You can find me at in katy-sidorova

# What are we going to DO in this workshop?

**Understand what is IAST.**

Understand what Interactive Application Security Testing is and what information it can provide.

**Learn how IAST tools work under the hood.**

Talk about how instrumentation serves a big role in building IAST tools.

**Learn How Instrumentation Works.**

**Write a Profiler.**

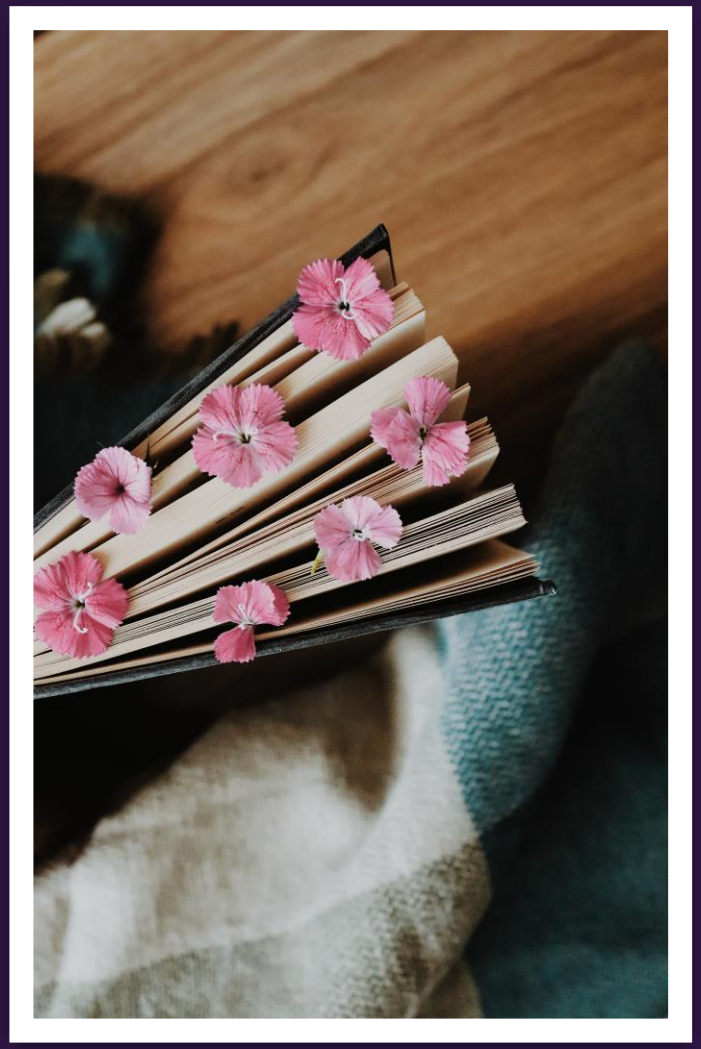
Build your own first simple profiler that instruments a log method.

**Solve a Riddle.**

Use a profiler to solve a riddle.

# Interactive Application Security Testing (IAST)

… is an application security testing method that tests the application while the app is run by an automated test, human tester, or any activity “interacting” with the application functionality.



# Methodologies for Application Security Testing

## SAST
**Static Application Security Testing**

Examine source code in a non-runtime environment early in the SDLC.

## IAST
**Interactive Application Security Testing**

Tests the application while the application is running.

## DAST
**Dynamic Application Security Testing**

Tests the applications from the exterior like a black-box scanner

# Instrumentation – Adding functionality to another application

```
public void doSomething(String[] arguments)
{
  logger.log(Level.INFO, "Staring DoSomething with" + arguments.length + " args");

  // ... do somethnig ...

  logger.log(Level.INFO, "Finished DoSomething with" + arguments.length + " args");
}
```

```
private static long totalTime = 0;
public void doSomething(String[] arguments)
{
  long startTime = System.currentTimeMillis();
  logger.log(Level.INFO, "Staring DoSomething with" + arguments.length + " args");
  long opTime = System.currentTimeMillis() - startTime;
  totalTime += opTime;

  // ... do somethnig ...

  startTime = System.currentTimeMillis();
  logger.log(Level.INFO, "Finished DoSomething with" + arguments.length + " args");
  opTime = System.currentTimeMillis() - startTime;
  totalTime += opTime;
}
```

# IAST and Profilers use instrumentation for their work

**Interactive Application Security Testing**

Gather user input, database access, system calls, logging, etc.

**Performance profilers**

Gather memory allocations, time spent in methods, etc.

# Other applications of Instrumentation

**Event logging and tracing**

**Event monitoring**

**Debugging in production**

# What are we going to DO in this workshop?

**Understand what is IAST.**

Understand what Interactive Application Security Testing is and what information it can provide.

**Learn how IAST tools work under the hood.**

Talk about how instrumentation serves a big role in building IAST tools.

**Learn How Instrumentation Works.**

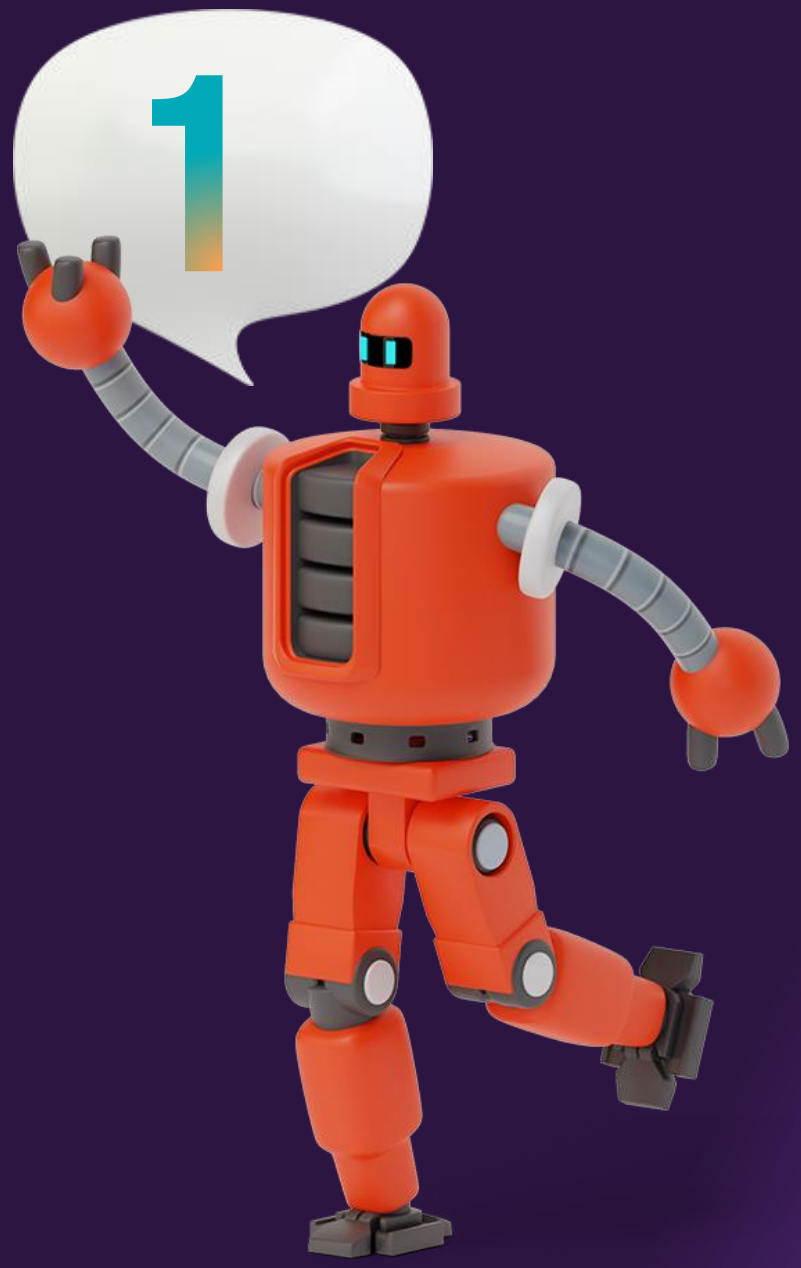
**Write a Profiler.**

Build your own first simple profiler that instruments a log method.

**Solve a Riddle.**

Use a profiler to solve a riddle.

# Instrumentation in java – how does it work

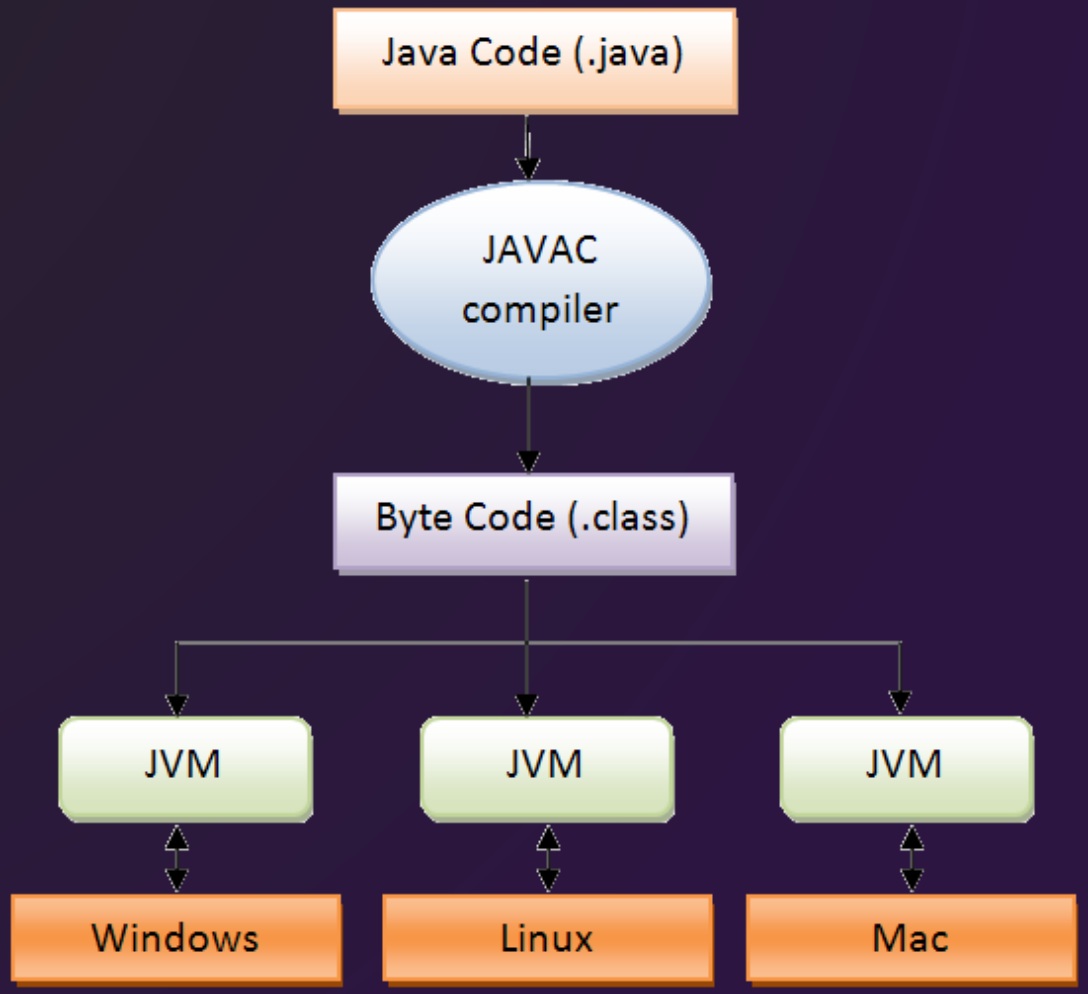


# In java There is a special API for Instrumentation

**Java instrumentation API - java.lang.instrument**

- ASM
- Javassist
- Byte Buddy

# Jvm – java code is compiled and converted to .class



# Jvm – java code is compiled and converted to .class

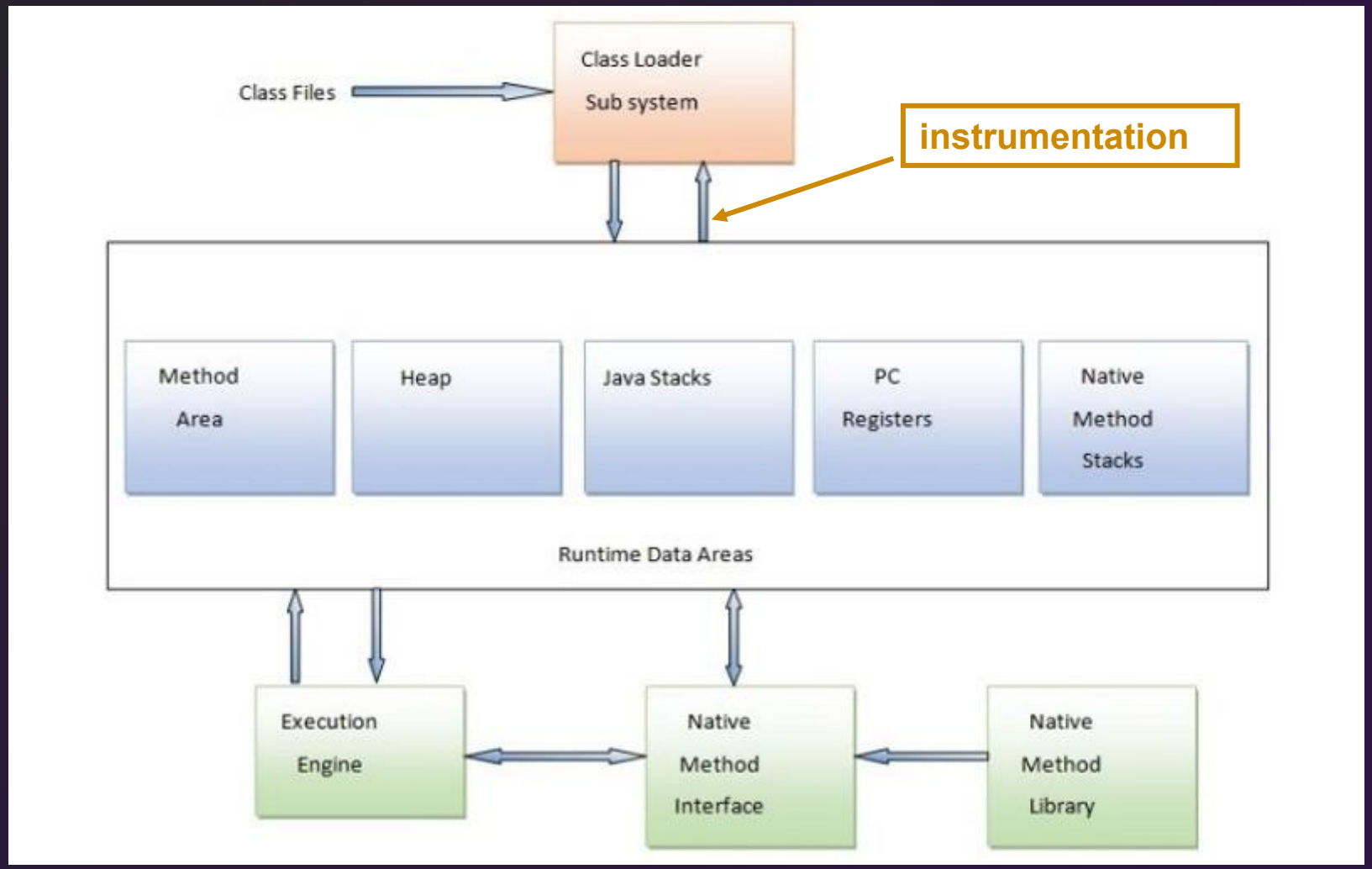
javap –v

javap –v java.lang.Object

```
// access flags 0xA
private static printGreeting()V
 L0
  LINENUMBER 7 L0
  GETSTATIC java/lang/System.out : Ljava/io/PrintStream;
  LDC "Hello, World!"
  INVOKEVIRTUAL java/io/PrintStream.println (Ljava/lang/String;)V
 L1
  LINENUMBER 8 L1
  RETURN
  MAXSTACK = 2
  MAXLOCALS = 0
```

**javap –v ./HelloWorld/out/production/helloWorld/com/codefest/testapp/HelloWorld.class**

# JVM - the JVM loads the bytecode



# Instrumentation API –
# as classes are loaded the jvm notifies transformers

**Add Class File Transformers to Instrumentation**

Before application classes are loaded: Class File Transformers are added to the instrumentation.

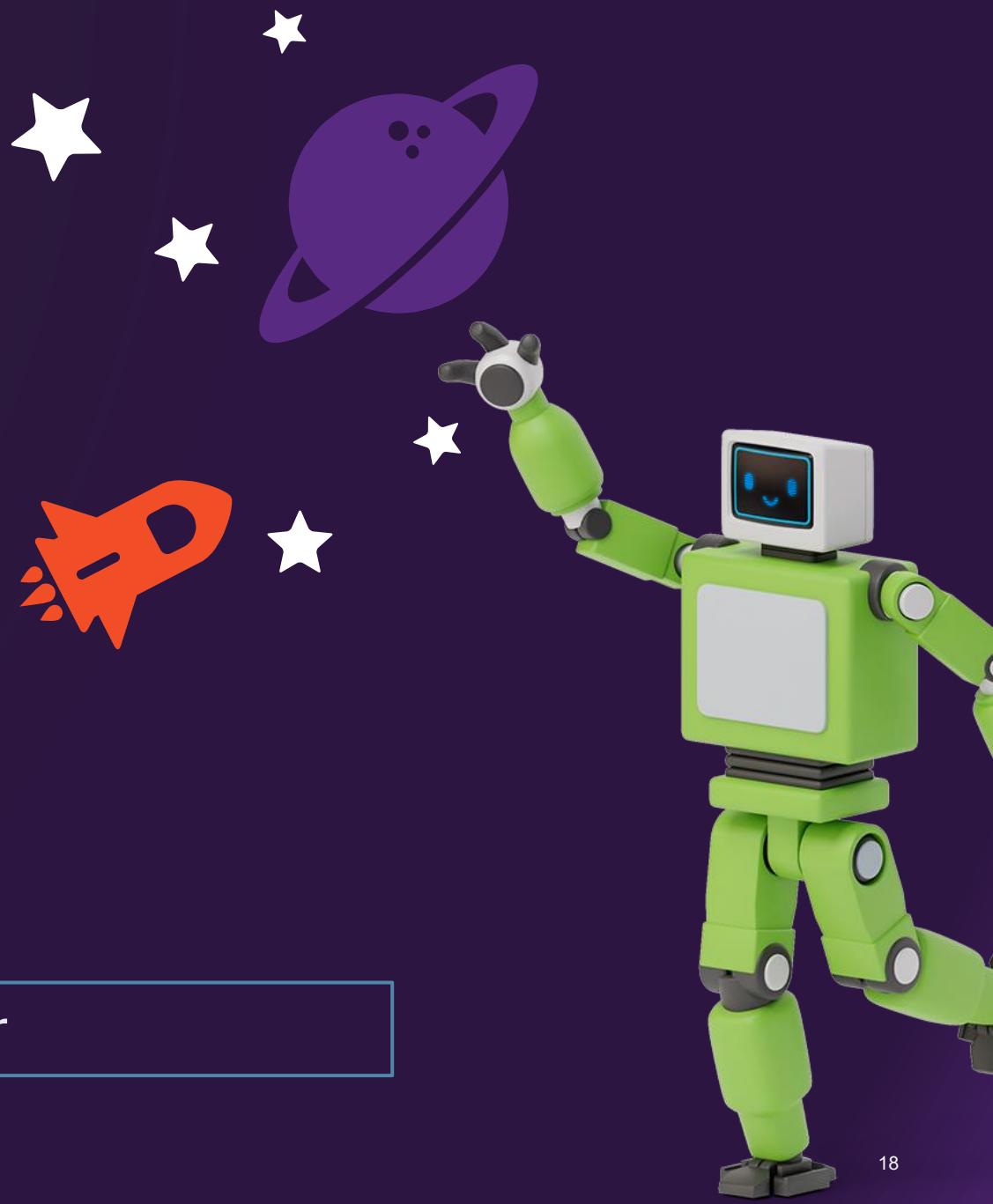
**As classes are loaded the transformers are notified**

When an application class is loaded, all transformers get notified that the class is loaded

**Class File Transformers change the classes**

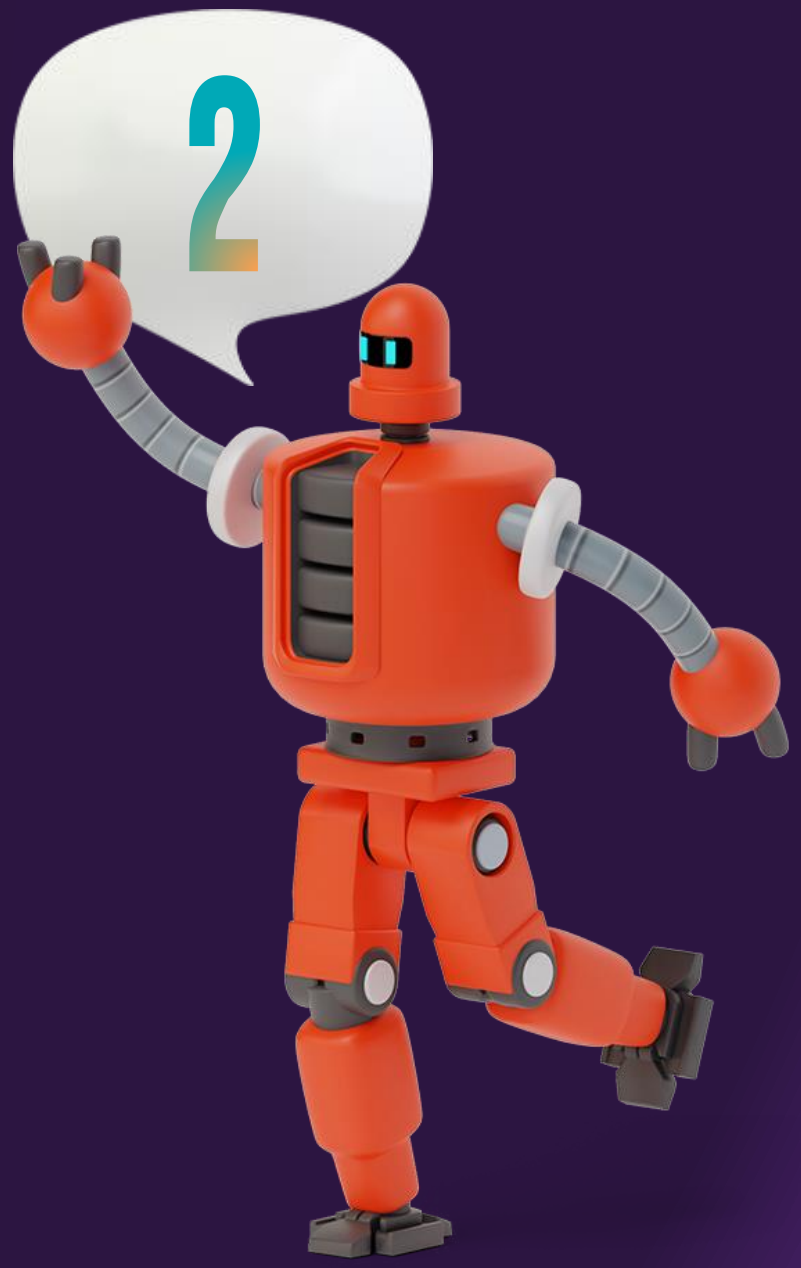
When a transformer is called with a class (bytecode) it has the option to change it.

# INSTRUMENTATION IS PERFORMED BY A JAVA AGENT



```
java -javaagent:agent.jar -jar application.jar
```

# Instrumentation in java – how it looks in the code



# CALCULATE THE TIME SPEND IN A LOG METHOD

```
for (int i = 0; i < 5; i++)
{
  logger.log(Level.INFO, "My log message");
}
```

```
public void log(Level level, String msg)
{
  if (!isLoggable(level))
  {
    return;
  }
  LogRecord lr = new LogRecord(level, msg);
  doLog(lr);
}
```

# CALCULATE THE TIME SPEND IN A LOG METHOD

```
for (int i = 0; i < 5; i++)
{
  logger.log(Level.INFO, "My log message");
}
```

```
public void log(Level level, String msg)
{
  long startTime = System.currentTimeMillis();
  if (!isLoggable(level))
  {
    return;
  }
  LogRecord lr = new LogRecord(level, msg);
  doLog(lr);
  long endTime = System.currentTimeMillis();
  long opTime = endTime - startTime;
  System.out.println("operation time: " + opTime + " miliseconds!");
}
```

# HOW DOES IT LOOK IN THE CODE – PREMAIN AND AGENTMAIN

```
public static void premain(String agentArgs, Instrumentation inst)
{
  transformClass(inst);
}
```

```
private static void transformClass(Instrumentation instrumentation)
{
  ClassFileTransformer logTransformer = new LogTransformer();
  instrumentation.addTransformer(logTransformer, true);
}
```

# HOW DOES IT LOOK IN THE CODE – CLASS FILE TRANSFORMER

```
private static void transformClass(Instrumentation instrumentation)
{
  ClassFileTransformer logTransformer = new LogTransformer();
  instrumentation.addTransformer(logTransformer, true);
}
```

```
public class LogTransformer implements ClassFileTransformer
 {
   ...

   public byte[] transform(ClassLoader loader, String className, Class<?> classBeingRedefined,
                           ProtectionDomain protectionDomain, byte[] classfileBuffer)
   {
     ...
   }
```

# CALCULATE THE TIME SPEND IN A LOG METHOD

```
public void log(Level level, String msg)
{
  long startTime = System.currentTimeMillis();
  if (!isLoggable(level))
  {
    return;
  }
  LogRecord lr = new LogRecord(level, msg);
  doLog(lr);
  long endTime = System.currentTimeMillis();
  long opTime = endTime - startTime;
  System.out.println("operation time: " + opTime + " miliseconds!");
}
```

```java
public byte[] transform(ClassLoader loader, String className, Class<?> classBeingRedefined,
                        ProtectionDomain protectionDomain, byte[] classfileBuffer)
{
  byte[] byteCode = classfileBuffer;

  if (!className.equals(CLASS_TO_INSTRUMENT))
  {
    return byteCode;
  }

  try
  {
    ClassPool cp = ClassPool.getDefault();
    CtClass cc = cp.get(CLASS_TO_INSTRUMENT);
    CtMethod method = cc.getDeclaredMethod(TRANSFORM_METHOD);

    if (method == null)
    {
      return byteCode;
    }

    // Add code to the beginning of a method
    method.addLocalVariable("startTime", CtClass.longType);
    method.insertBefore( "startTime = System.currentTimeMillis();");

    // Add code to the end of a method
    StringBuilder endBlock = new StringBuilder();
    method.addLocalVariable("endTime", CtClass.longType);
    method.addLocalVariable("opTime", CtClass.longType);
    endBlock.append("endTime = System.currentTimeMillis();");
    endBlock.append("opTime = endTime - startTime;");
    endBlock.append("System.out.println(\"[Added by Agent] operation time: \" + opTime + \"
miliseconds!\");");
    method.insertAfter(endBlock.toString());

    byteCode = cc.toBytecode();
    cc.detach();
  }
  catch (Exception e)
  {
  }

  return byteCode;
}
```

```java
String CLASS_TO_INSTRUMENT =
  "java/util/logging/Logger";


String TRANSFORM_METHOD = "log";
```

# HOW DOES IT LOOK IN THE CODE

```
public byte[] transform(ClassLoader loader, String className, Class<?> classBeingRedefined,
                        ProtectionDomain protectionDomain, byte[] classfileBuffer)
{
  byte[] byteCode = classfileBuffer;

  if (!className.equals(CLASS_TO_INSTRUMENT))
  {
    return byteCode;
  }
```

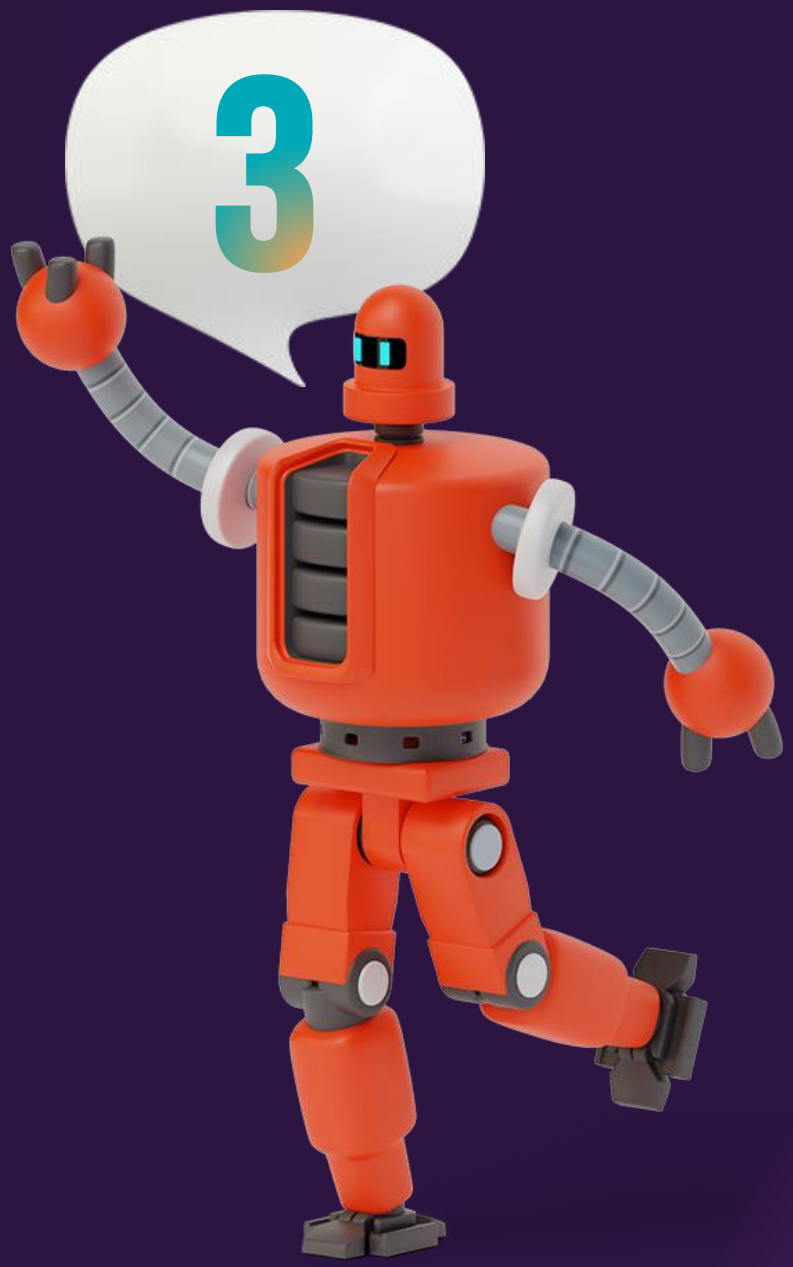
# HOW DOES IT LOOK IN THE CODE

```
try
{
  ClassPool cp = ClassPool.getDefault();
  CtClass cc = cp.get(CLASS_TO_INSTRUMENT);
  CtMethod method = cc.getDeclaredMethod(TRANSFORM_METHOD);

  if (method == null)
  {
    return byteCode;
  }
```

# HOW DOES IT LOOK IN THE CODE – USING JAVASSIST

```
    CtMethod method = cc.getDeclaredMethod(TRANSFORM_METHOD);

    if (method == null)
    {
      return byteCode;
    }

    // Add code to the beginning of a method
    method.addLocalVariable("startTime", CtClass.longType);
    method.insertBefore("startTime = System.currentTimeMillis();");

    // Add code to the end of a method
    StringBuilder endBlock = new StringBuilder();
    method.addLocalVariable("endTime", CtClass.longType);
    method.addLocalVariable("opTime", CtClass.longType);
    endBlock.append("endTime = System.currentTimeMillis();");
    endBlock.append("opTime = endTime - startTime;");
    endBlock.append("System.out.println(\"[Added by Agent] operation time: \" + opTime + \"
miliseconds!\");");
    method.insertAfter(endBlock.toString());

    byteCode = cc.toBytecode();
    cc.detach();
```

# Instrumentation in java – how to package and run



# RUNNING A JAVA AGENT

```
-javaagent:<path/to/agent.jar>
```

```
java -jar application.jar
```

```
java -javaagent:agent.jar -jar application.jar
```

# JAVAAGENT – IT IS ALSO IMPORTANT TO PACKAGE YOUR DEPENDENCIES

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-assembly-plugin</artifactId>
      <version>3.1.1</version>

      <configuration>
        <descriptorRefs>
          <descriptorRef>jar-with-dependencies</descriptorRef>
        </descriptorRefs>
        <archive>
          <manifestEntries>
            ...
          </manifestEntries>
        </archive>
      </configuration>

      <executions>
        <execution>
          <id>make-assembly</id>
          <phase>package</phase>
          <goals>
            <goal>single</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

# JAVAAGENT – DECLARATION IN THE MANIFEST

Java agent is a simple jar with two main features:

1. Agent class must implement agentmain and premain
2. Menifest file – to allow instrumentation and change of classes

MANIFEST.MF - Notepad

File Edit Format View Help

```
Manifest-Version: 1.0
Premain-Class: irach.demo.profiler.Profiler
Built-By: ira
Agent-Class: irach.demo.profiler.Profiler
Can-Redefine-Classes: true
Can-Retransform-Classes: true
Created-By: Apache Maven 3.8.1
Build-Jdk: 1.8.0_242
Boot-Class-Path: profiler-1.0-jar-with-dependencies.jar
Main-Class: irach.demo.profiler.Profiler
```

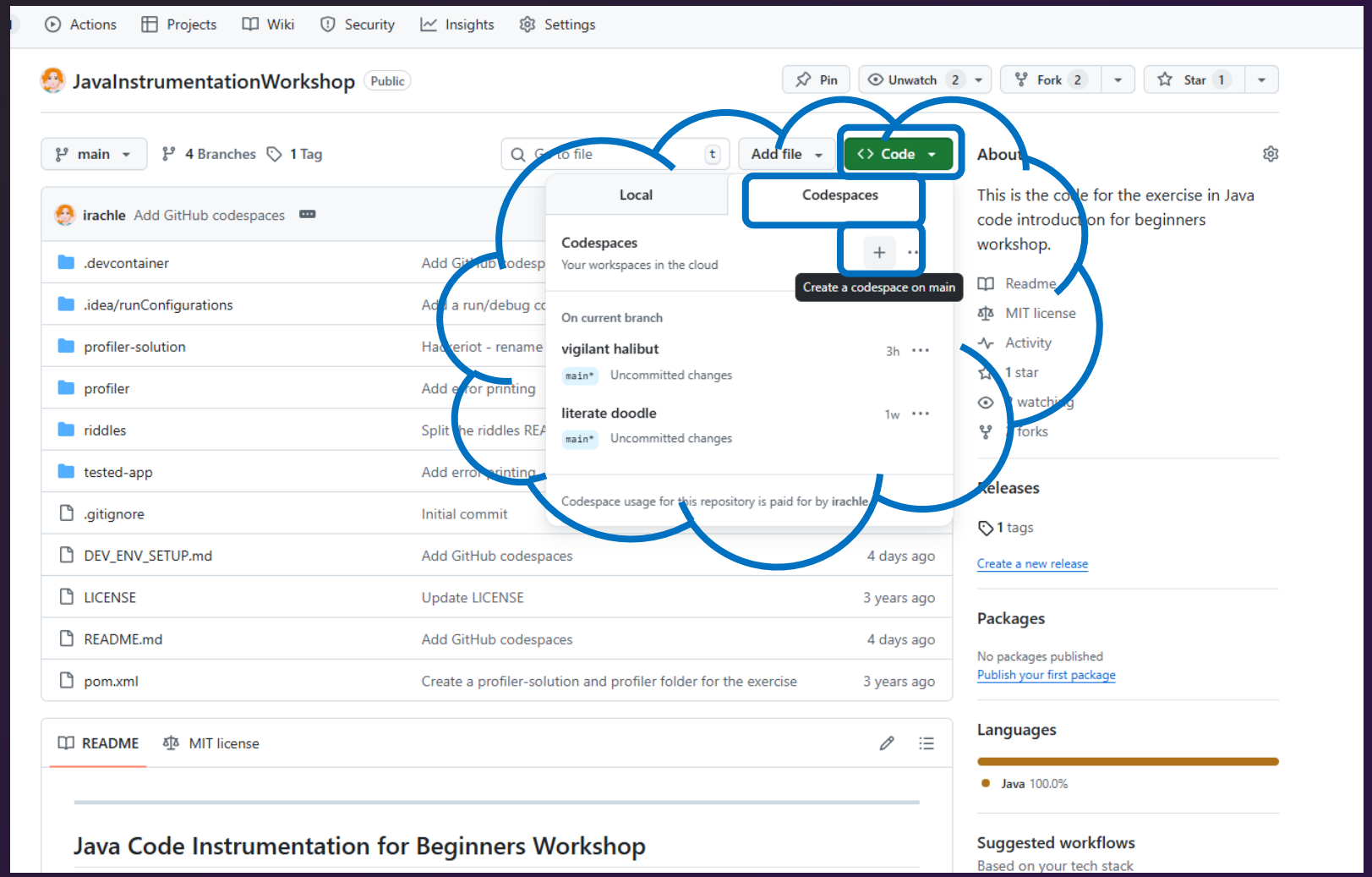
# JAVAAGENT – DECLARATION IN THE MANIFEST

```
<configuration>
  <descriptorRefs>
      <descriptorRef>jar-with-dependencies</descriptorRef>
  </descriptorRefs>
  <archive>
      <manifestEntries>
        <Main-Class>irach.demo.profiler.Profiler</Main-Class>
        <Premain-Class>irach.demo.profiler.Profiler</Premain-Class>
        <Agent-Class>irach.demo.profiler.Profiler</Agent-Class>
        <Can-Redefine-Classes>true</Can-Redefine-Classes>
        <Can-Retransform-Classes>true</Can-Retransform-Classes>
        <Boot-Class-Path>profiler-1.0-jar-with-dependencies.jar</Boot-Class-Path>
      </manifestEntries>
  </archive>
</configuration
```

```
.profiler.Profiler

Agent-Class: irach.demo.profiler.Profiler
Can-Redefine-Classes: true
Can-Retransform-Classes: true
Created-By: Apache Maven 3.8.1
Build-Jdk: 1.8.0_242
Boot-Class-Path: profiler-1.0-jar-with-dependencies.jar
Main-Class: irach.demo.profiler.Profiler
```

# Exercise – Now it’s your turn

- Create a new Codespace in GitHub  from the following repository: https://github.com/irachle/JavaInstrumentationWorkshop




SCAN ME

# Exercise – Now it’s your turn

JavaInstrumentationWorkshop [Codespaces: vigilant halibut]

EXPLORER

JAVAINSTRUMENTATIONWORKSHOP [CODESPACE...

- .devcontainer
- .idea
- .vscode
- profiler
- profiler-solution
- riddles
- tested-app
- .gitignore
- DEV_ENV_SETUP.md
- LICENSE
- pom.xml
- README.md

[Preview] README.md

If you don't have a preferred working IDE, download and install Intellij **Community** Edition for free.

2. Maven
   Download Maven, the build tool used in this project, from here and extract it to a folder (no installation required). Then, add the maven/build folder to the PATH variable.
3. JDK
   If you don't have a JDK installed, download any version of JDK. The project was tested with JDK 1.8, 17 and 23. You can use IntelliJ to download the JDK by following the steps explained here.

## Test and run the profiler

Build the project

```
mvn install
```

To execute the tested app without the profiler run

```
java -jar ./tested-app/target/tested-app-1.0.jar
```

To execute the tested app with the profiler run

```
java -javaagent:./profiler/target/profiler-1.0-jar-with-dependencies.jar -jar ./tested-app/target/tested-app-1.0.jar
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
@irachle ➜ /workspaces/JavaInstrumentationWorkshop (main) $ Open chat ctrl+I. Start typing to dismiss.
```

OUTLINE
TIMELINE
JAVA PROJECTS
MAVEN

SCAN ME

# Exercise – Now it's your turn

- Create a new Codespace in GitHub from the following repository: https://github.com/irachle/JavaInstrumentationWorkshop

- **Exercise**:
  - Run the app twice: once without the profiler and once with it.
  - **By changing the profiler add a printed "Hello" line after operation time calculation**
  - Change the profiler to print a log message.
  - Change the profiler to count the number of times the log method was called.
- **Project structure:**
  - Tested app – simple application to run with your profiler.
  - Profiler – a simple profiler as discussed here. You will use this folder to write the next steps of your profiler.
  - Profiler-solution – solution for the next steps.
  - Riddles folder – A folder with 2 riddle applications.

# Exercise – Now it's your turn

- Create a new Codespace in GitHub from the following repository: https://github.com/irachle/JavaInstrumentationWorkshop

- **Exercise**:
  - Run the app twice: once without the profiler and once with it.
  - **By changing the profiler add a printed "Hello" line after operation time calculation**
  - Change the profiler to print a log message.
  - Change the profiler to count the number of times the log method was called.

SCAN ME

- **Commands:**

```
> mvn install
> java -jar ./tested-app/target/tested-app-1.0.jar
> java -javaagent:./profiler/target/profiler-1.0-jar-with-dependencies.jar -jar ./tested-app/target/tested-app-1.0.jar
```

# HOW WILL THIS LOOK

```
java -jar ./tested-app/target/tested-app-1.0.jar
```

```
Sep 07, 2025 3:29:53 PM irach.demo.tested_app.JustApp main
INFO: My log message
Sep 07, 2025 3:29:53 PM irach.demo.tested_app.JustApp main
INFO: My log message
Sep 07, 2025 3:29:53 PM irach.demo.tested_app.JustApp main
INFO: My log message
Sep 07, 2025 3:29:53 PM irach.demo.tested_app.JustApp main
INFO: My log message
Sep 07, 2025 3:29:53 PM irach.demo.tested_app.JustApp main
INFO: My log message
```

# HOW WILL THIS LOOK

```
java -javaagent:./profiler/target/profiler-1.0-jar-with-dependencies.jar -jar ./tested-app/target/tested-app-1.0.jar
```

```
Sep 07, 2025 3:34:19 PM irach.demo.tested_app.JustApp main
INFO: My log message
[Added by Agent] operation time: 80 miliseconds!
Sep 07, 2025 3:34:19 PM irach.demo.tested_app.JustApp main
INFO: My log message
[Added by Agent] operation time: 1 miliseconds!
Sep 07, 2025 3:34:19 PM irach.demo.tested_app.JustApp main
INFO: My log message
[Added by Agent] operation time: 1 miliseconds!
Sep 07, 2025 3:34:19 PM irach.demo.tested_app.JustApp main
INFO: My log message
[Added by Agent] operation time: 0 miliseconds!
Sep 07, 2025 3:34:19 PM irach.demo.tested_app.JustApp main
INFO: My log message
[Added by Agent] operation time: 1 miliseconds!
```

# Recap

**Understand what is IAST.**

Understand what Interactive Application Security Testing is and what information it can provide.

**Learn how IAST tools work under the hood.**

Talk about how instrumentation serves a big role in building IAST tools.

**Learn How Instrumentation Works.**

**Write a Profiler.**

Build your own first simple profiler that instruments a log method.

**Solve a Riddle.**

Use a profiler to solve a riddle.

# Recap

**We wrote a java agent that instruments a log method, and adds some lines of code before doing its original job.**

But we want to collect data, to analyze it, so let's see if the agent can access parameters.

**How can this be done?**

# ACCESS PARAMETERS OF THE METHOD USING JAVA ASSIST?

Parameters start with $:

$0 – this

$1 – 1st parameter

$2 – 2nd parameter

…

# Exercise – Now it's your turn

- Create a new Codespace in GitHub  from the following repository: https://github.com/irachle/JavaInstrumentationWorkshop

- **Exercise**:
  - Run the app twice: once without the profiler and once with it.
  - By changing the profiler add a printed "Hello" line after operation time calculation
  - **Change the profiler to print a log message.**
  - Change the profiler to count the number of times the log method was called.

- **Commands:**

```
> mvn install
> java -jar ./tested-app/target/tested-app-1.0.jar
> java -javaagent:./profiler/target/profiler-1.0-jar-with-dependencies.jar -jar ./tested-app/target/tested-app-1.0.jar
```

SCAN ME

# GETTING THE LOG MESSAGE

- Simply printing the first parameter gives a text with:
  java.util.logging.LogRecord@66d3c617

- Possible solutions:
  - Find a way to get the log message from LogRecord
  - Find exactly the log method that is used by using also the method parameters
  - Instrument all log method overloads.

# GETTING THE LOG MESSAGE

```
public byte[] transform(ClassLoader loader, String className, Class<?> classBeingRedefined,
                        ProtectionDomain protectionDomain, byte[] classfileBuffer)
{
  byte[] byteCode = classfileBuffer;

  if (!className.equals(CLASS_TO_INSTRUMENT.replaceAll("\\.", "/")))
  {
      return byteCode;
  }

  ClassPool cp = ClassPool.getDefault();
  CtClass cc = cp.get(CLASS_TO_INSTRUMENT);
  CtMethod method = cc.getDeclaredMethod(TRANSFORM_METHOD,
      new CtClass[]{cp.get(TRANSFORM_METHOD_PARAM1), cp.get(TRANSFORM_METHOD_PARAM2)});

  if (method == null)
  {
    return byteCode;
  }

  // Add code to the beginning of a method
  method.insertBefore( "System.out.println(\"[Added by Agent] log message is: \" + $2);");

  byteCode = cc.toBytecode();
  cc.detach();
  return byteCode;
}
```

# Exercise – Solve a riddle

The riddle folder contains two java apps.

Run them and find the correct password.

Hint: You'll need a profiler that can read the log messages of the application.

- **Exercise**:
  - Run the first riddle – StartHere-1.jar
    Can you guess the correct password?
  - Change the profiler to print the logs message.
    Note: The logging method that is used by the riddles is the same one used by tested-app.
  - Run the first riddle with your new profiler. Can you get the correct password now?
  - Now solve the second riddle – ShiftNext-2.jar

SCAN ME

# Recap

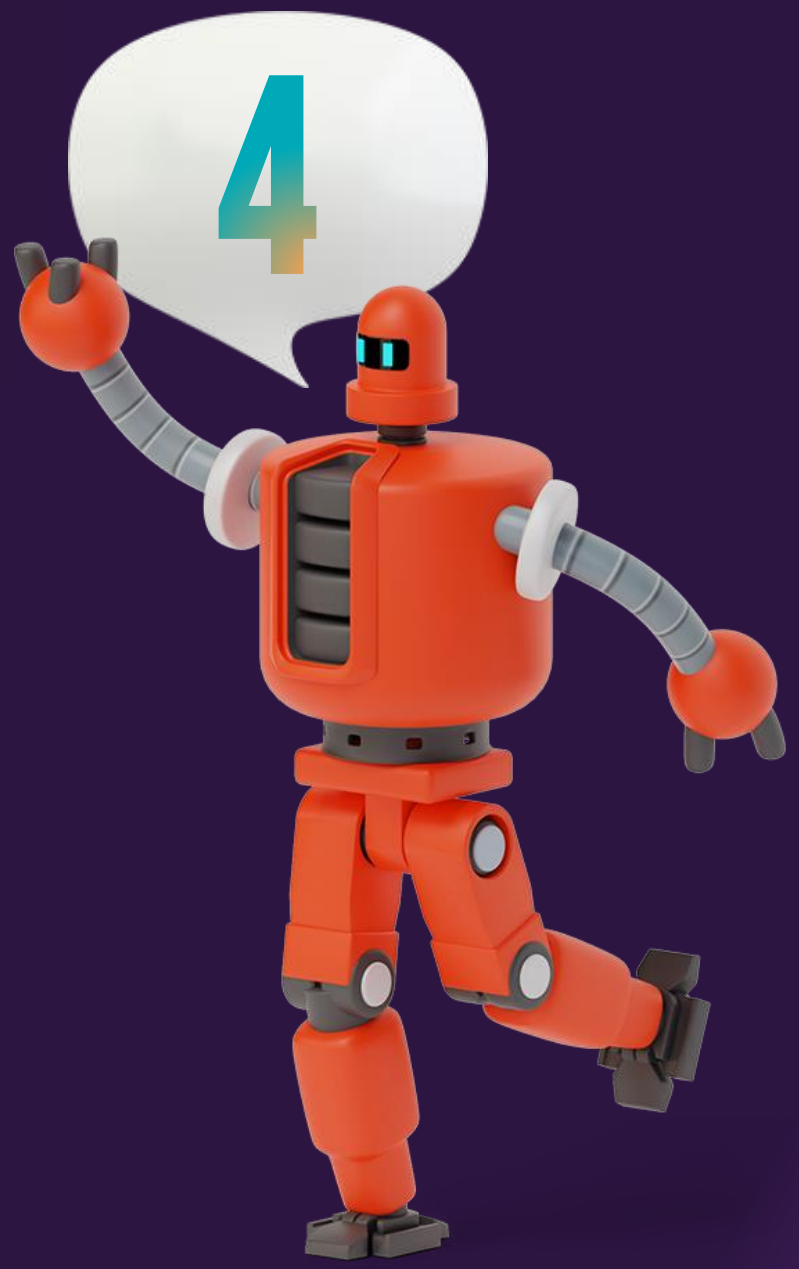
**We wrote a java agent that instruments a log method, and adds some lines of code before doing its original job.**

But we want to collect data, to analyze it, to save some state / statistics.

**How can this be done?**

# Collecting data and analyzing it.

creating a class in the profiler and sending information to it.



# CALLING A METHOD INSIDE A PROFILER CLASS

```
public class LogProfiler
{
  public static final LogProfiler $_INSTANCE = new LogProfiler();

  int count = 0;

  public void logMethodCalled()
  {
    count++;
    System.out.println("[Agent] log method was called " + count + " times");
  }
}
```

**The instance or method should be static.**

**Use this method to save state, gather statistics or check arguments.**

# CALLING A METHOD INSIDE A PROFILER CLASS

```
public byte[] transform(ClassLoader loader, String className, Class<?> classBeingRedefined,
                        ProtectionDomain protectionDomain, byte[] classfileBuffer)
{
  byte[] byteCode = classfileBuffer;

  if (!className.equals(CLASS_TO_INSTRUMENT.replaceAll("\\.", "/")))
  {
    return byteCode;
  }

  ClassPool cp = ClassPool.getDefault();
  CtClass cc = cp.get(CLASS_TO_INSTRUMENT);
  CtMethod method = cc.getDeclaredMethod(TRANSFORM_METHOD);

  if (method == null)
  {
    return byteCode;
  }

  method.insertAfter("irach.demo.profiler_solution.LogProfiler.$_INSTANCE.logMethodCalled();");

  byteCode = cc.toBytecode();
  cc.detach();

  return byteCode;
}
```

# Exercise – Now it's your turn

- Create a new Codespace in GitHub  from the following repository: https://github.com/irachle/JavaInstrumentationWorkshop

- **Exercise**:
  - Run the app twice: once without the profiler and once with it.
  - By changing the profiler add a printed "Hello" line after operation time calculation
  - Change the profiler to print a log message.
  - **Change the profiler to count the number of times the log method was called.**

- **Commands:**

```
> mvn install
> java -jar ./tested-app/target/tested-app-1.0.jar
> java -javaagent:./profiler/target/profiler-1.0-jar-with-dependencies.jar -jar ./tested-app/target/tested-app-1.0.jar
```

SCAN ME

# Exercise – Now it's your turn

- Create a new Codespace in GitHub  from the following repository: https://github.com/irachle/JavaInstrumentationWorkshop

- **Exercise**:
  - Run the app twice: once without the profiler and once with it.
  - By changing the profiler add a printed "Hello" line after operation time calculation
  - Change the profiler to print a log message.
  - **Update the profiler to use a static class that inspects log messages for passwords.**

SCAN ME

# Summary - Let’s review some concepts

**IAST**
**Interactive Application Security Testing**

Tests the application while the application is running.

**IAST**
**Interactive Application Security Testing**

Gather information on real requests, and pinpoints vulnerable code location.

**Instrumentation**
**Adding functionality on the fly**

Used by IAST, performance profilers, loggers, event monitors

**Instrumentation API and frameworks**

**Implemented a simple profiler**

That instrumented the log method.

**Solved a riddle**

Using the profiler you wrote

BLACKDUCK®

# WE VALUE YOUR FEEDBACK.

## Thank You

You can find us at in or f ira-cherkes-levinshteyn or in katy-sidorova